Logic and Computation Notes

William Traub

Contents

1 1/7, Defining Functions **3**
1.1 Course Plan 3

2 1/8 Functional Programming **4**
2.1 OCaml notes 4

3 1/12 **5**
3.1 Functions 5
3.2 Specification 5

4 1/14 Pattern Matching and Data **7**
4.1 Functions 7
4.2 ADT's 8

5 1/15 Recap and Structural Recursion **10**
5.1 Recursive types 10

6 1/21 - Specification cont. **13**

7 1/22 - Formalization, Coding and Bug Hunting **14**

8 Type Inference, Modules **15**

9 2/4 **16**

10 Invariants and Generalizing Modules **18**

11 Functors and Mutability **19**
11.1 Functors, cont. 19
11.2 Mutability 20

12 References and Aliasing **22**

13 2/25 **24**

14 3/9 **25**
14.1 What is a program? 25
14.2 Creating a Calculator Language 25

15 3/12 - Programming Languages, Cont. **27**

16 3/16 - Intro to Type Checking **28**
16.1 What is a Type 28

17 3/23 - Compiling **29**

18 Formalizing Compiler Correctness and Failiures of PBT **31**

19 4/1 - Compiler Optimization **32**

1 1/7, Defining Functions

mk-requests:(ListID)
$\forall S, \forall r, \forall a, \exists R,$ mk-requests$(S, r, a) = R$
$\forall S, \forall r, \forall a, \forall R,$ mk-requests$(S, r, a) = R \implies \Sigma_{(\text{fr, to, amt})} amt = a$
$\forall S, \forall r, \forall a, a < 0 \implies \nexists R,$ mk-requests$(S, r, a) = R$
Advantages of proofs:

- Computable - we can turn these statements into tests and proofs
- Independent - decoupled from the code itself. We don't need to worry about runtime in order to test.

Coding a property?
Because we are universally quantifying, we would generally need to enumerate over the entire set of each $\forall$
Write a testing harness to substitute for the enumeration. Use randomness to generate test data.

1.1 Course Plan

Modules:

1. Programming and PBT
2. Engineering with Abstractions
3. Analyzing Programming Languages

2 1/8 Functional Programming

What is functional programming?
When we write software and it gets complicated, it helps to break it down into smaller pieces. With functional programming, we get 'glue' where you are composing functions with each other.
Goal: get us to write a fizzbuzz function that takes in an int and gives a fizzbuzz string output

```
function fizzbuzz:
  input n number
  output string

  if (n % 3) = (n % 5) = 0 return "fizzbuzz"
  else if n % 3 = 0 return "fizz"
  else if n % 5 = 0 return "buzz"
  else return n as a string
```

2.1 OCaml notes

- in the REPL (utop) terminate expressions with ;;
- standard arithmetic, carrot is string concat
- must use +. to add floats
- use float of int to cast float to int
- double quotes for string
- not binds tighter than other
- if false then 2 else 3 (ternerary must have same type)
- comparison operators, single =
- let x = 4 in x / 2
- overwriting variables instead of mutating them
-
```
let <var> = <expr> in <body> ;;
< store, let <var> = <expr>> -> <store[<var> ;;
```

3 1/12

```
let x = 19 in let x = x < 10 in x ;;
(* goes to *)


let x = 3 in x + x ;; (* -> 6 *)
let x = 4 in let y = 5 in x * y ;; (* 20 *)
let x = 19 in (let x = x < 10 in x) ;;
```

3.1 Functions

Functions are boring and everywhere.

```
(* anonymous function *)
let f = fun (x : int) -> x + 1 ;;
f : int -> int -> int
```

OCaml is right-associative so

```
let f = fun (x : int) -> fun (y : int) -> x * y ;;
let f (x : int) (y : int) = x * y ;;
```

These functions are isomorphic and will result in about the same thing.

```
let f (g : int -> int) (y : int) = g y;;
let adder = f (fun (y : int) )
```

Lexical Scope

```
let y   = 1 in
let f x = x + y in
let y   = 2 in
f 3 ;;
```

There is closure used where the function substitutes y into its body and just becomes f(x) = x + 1.

3.2 Specification

We have high-order functions that can take in other functions.

```
let max3 (i : int) (j : int) (k : int) =
  if i >= j
  then if i >= k
    then i
    else k
  else if l >= k
    then j
    else k
```

We want to specify the function in formal logic.

$$\forall i, j, k \in \mathbb{Z}$$

$$max3(i, j, k) \geq i \wedge$$
$$max3(i, j, k) \geq j \wedge$$
$$max3(i, j, k) \geq k \wedge$$
$$max3(i, j, k) \in \{i, j, k\}$$

4 1/14 Pattern Matching and Data

4.1 Functions

```
fun (x : int -> string) -> (string -> string) (z : int)
```

Functions are everything. You can think of let bindings as function application.

```
let x = 19 in
let y = x < 10
in y ;;
```

```
(fun (x) ->
  (fun (y) -> y
  ) (x < 10)
) 19
```

What is pattern matching?

```
let show (x : int) =
  if x = 1 then "one" else
  if x = 2 then "two" else
  if x = 3 then "three" else
  "> three";;

let show (x : int) =
  match x with
  | 1 -> "one"
  | 2 -> "two"
  | 3 -> "three"
  | _ -> "> three";;
```

Because we have typing, we can more aggresively optimize. `https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/`
In OCaml, we have pairs or tuples. Always use pattern matching to unfold or destruct.

```
let student_of_2800 (name_stu : string * bool) : bool =
match (name, is_student) with
| (_, false) -> false
| (_, true) -> true ;;
```

You can have a nested match but you must wrap with parenthesis.

```
let shift_x (pt : int * int) (delta : int) : int * int =
  match pt with
    | (x, y) -> (x + delta, y)
    ;;

let shift_x ((x, y) : int * int) (delta : int) : int * int =
  (x + delta, y)
  ;;
```

Definition 1. *Sometimes we have basic types but sometimes we want to provide a light form of documentation. We can write a* ***type alias*** *to define our own.*

```
type point2d = int * int;;
let shift_x (point2d : int * int) (dx, dy : int * int) :
   point2d =
  match (xy, dxdy) with | (x, y), (dx, dy) -> (x + dx, y + dy)
     ;;
```

4.2 ADT's

Definition 2. *And* ***Algebraic Data Type (ADT)*** *is a composite data type. This is similar to a union or Enum. An example of a sumtype.*

```
type weekday =
  | Monday
  | Tuesday
  | Wednesday
  | Thursday
  | Friday ;;

type two_days = (weekday * weekday);;

let next_weekday (d : weekday) : weekday =
  match d with
  | Monday  -> Tuesday
  | Tuesday -> Wednesday
  | Wednesday -> Thursday
  | Thursday -> Friday
  | Friday -> Monday ;;

type my_bool =  | myTrue | myFalse ;;

type distance =
  | Inches of float
  | Feet of float
  | Yards of float ;;

let d1 = Inches 10. ;;
let d2 = Yards 12. ;;

let distance_to_inches (d : distance) : float =
match d with
  | Inches x -> x
  | Feet x -> x *. 12.
  | Yards x -> x *. 36. ;;

distance_to_inches d2 ;; (* returns )
```

```
type quantity =
  | Fraction of int * int
  | Float of float
  | Arbitrary of string ;;

let string_of_quantity (q : quantity) : string =
  match q with
  | Fraction (n, d) -> string_of_int n ^ "/" ^ string_of_int d
  | Float f -> string_of_float f
  | Arbitrary s -> s ;;
```

Record type in OCaml

```
type offset
  = { location : point2d ; delta distance * distance}

let small_offset = {location = (3, 3)
                    ; delta = (Float 3., Fraction (1, 2))}
let _ =
  match small_offset.delta with
    | (_, Fraction (_, _)) -> print_endline "fractional"
    | (_, Float _) -> print_endline "float"
    | (_, Arbitrary _) -> print_endline "arbitrary" ;;
```

5 1/15 Recap and Structural Recursion

```
type int_result =
  | IntOk of int
  | IntError of string ;;

type person = { name : string ; age : int; registered : bool} ;;
let alice = { name = "alice" ; age = 24 ; registered = true } ;;

let string_of_person (p : person) : string =
  p.name ^ " is " string_of_int p.age ;;

let other_string_of_person { name ; age = their_age ; _ } :
   string =
  name ^ " is " string_of_int their_age ;;
```

The million dollar mistake: nullity. Can be solved by options.

```
type int_option =
  | IntOpt of int
  | IntOptEmpty
```

5.1 Recursive types

Lists in OCaml

```
  [1 ; 2 ; 3]
```

Statically typed, can only have single type.

```
  type int_list =
    | Empty
    | Cons of int * int_list

  let intlist1 = Cons (2, (Cons (1, Empty)))

  let rec int_with_list_sum (xs : int_list) : int =
    match xs with
    | Empty -> 0
    | Cons (x, xs') -> x + int_list_sum xs'

  let rec int_list_length (xs : int_list) : int =
    | Empty -> 0
    | Cons (_x, xs) -> 1 + int_list_length xs

  let rec int_list_add1 (xs : int_list) : int_list =
    | Empty -> Empty
    | Cons (x, xs) -> Cons (x * 2, int_list_add1 xs)

  let rec int_list_mul2 (xs : int list) : int list =
    | [] -> []
    | x::xs -> (x * 2) :: (int_list_mul2 xs)
```

We can create a high-order function map for this

```
let rec map (f : int -> int) (xs : int list) : int list =
  match xs with
  | [] -> []
  | x::xs -> (f x) :: (map f xs)

let add1 = map (fun (x : int) -> x + 1)
let mul2 = map (fun (x : int) -> x * 2)
```

We can reduce or fold to find the sum of a list. We are taking a data structure and an accumulator and apply a binary function to the acc and each element of the structure.

```
let rec fold ( f : int -> int -> int) (acc : int) (xs : int
   list) : int_list =
  match xs with
  | [] -> acc
  | (x::xs') ->
    let acc' = f acc x in
    fold f acc' xs'

let sum = fold (fun (acc : int) (x : int) -> acc + x) 0
let product = fold (fun (acc : int) (x : int) -> acc * x) 1

let rec append (xs : int list) (ys : int list) : int list =
  match xs with
  | [] -> ys
  | (x :: xs') -> x :: append xs' ys

let rec reverse (xs : int list): int list =
  match xs with
  | [] -> []
  | (x :: xs') -> append (reverse xs') [x]
```

Functions sum and reverse are commutative. $\forall x \in$ int list, sum(reverse(xs)) = sum(xs)
How to test a list function.
Generating a random list: start with deciding the length.

```
let rec mk_list_of_length (len : int) : int list =
  if len <= 0 then [] else
    let hd = Random_int 500 in
    let tl = mk_list_of_length (len - 1) in
    hd :: tl

let list_gen () : int list =
  let len = Random.int 300 in
  mk_list_of_length len
```

We need to serialize a counterexample to create a tester

```
let prop_sum_rev (sum L int list -> int) (rev : int list -> int
    list) : int list option =
let xs = list_gen () in
if sum (reverse xs) = sum xs then None else
  Some xs
```

```
let rec do_tests (sum : int list -> int) (rev : int list -> int
    list) (n : int) =
  if  n <= 0 then
    print_endline "no issues"
  else
    match (prop_sum_rev sum rev) with
    | None -> do_tests sum rev (n - 1)
    | Some s -> print_endline("found a bug: " ^ stringify s)
```

6 1/21 - Specification cont.

We might use a comment to annotate a function;

```
(* return the larger of the two numbers *)
let max (i : int) (j : int) = ...
```

This can be written as $S = \{P | P$ implements "return the larger of two numbers" $\}$
This is somewhat ambiguous, the set is infinite.

```
(* sorts the input list *)
let rec sort (xs : int list) = ...

type row = {
  name : string;
  gid : string; (* global id *)
  lid : string (* local id *)
}

let rec sort_rows (xs : row list) : row list = ...
```

It's unclear how this needs to be sorted.
A data type is its own specification. $S = \{P | P$ has int list $\rightarrow$ int list$\}$. This set is also infinite.
We want something more powerfull then the second while still expressing the meaning of the first.

1. $\forall xs, ys,$ sort xs $= ys \implies \forall i, j, 0 \leq i < j <$ length $ys \implies ys[i] \leq ys[j]$
2. $\forall x,\ x \in ys \wedge x \in xs$
   or $\forall xs, ys,$ sort xs = ys $\implies \forall i, |xs\{i\}| = |ys\{i\}|$

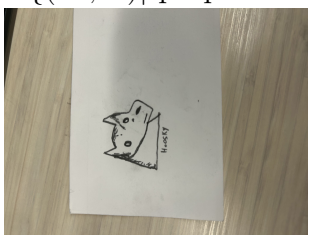
We can restate this as $S = \{P | \forall i, R(i, P(i))\}$.

7 1/22 - Formalization, Coding and Bug Hunting

Plan

- Formalize our sort specification
- Discuss computability
- Implement a the specification and find a bug

CODE3
We now have our implementation of our specification and can write a full test function.
CODE4
We take in our sort function as an input as we are validating if a certain function works. Because we are talking about our computable test function, we should also specify that.
$S = \{P | \forall xs,\ \text{prop-sort-correct}(xs, P)\}$ where prop-sort-correct is int list $\rightarrow$ bool.
This is an undecidable problem so we must modify it. $S = \{(xs, P) |\ \text{prop-sort-correct}(xs, P)\}$



We were told to draw a pretty picture so here is mine.
PBT recipie:

1. Write down our logical spec as a program
2. Use randomness to perform (directed) sampling of the input values
3. If no counterexample: increase confidence and reexamine the generator and proposition
4. If the is a counterexample, the prop is invalid

CODE5 List init is like a racket buildlist
We are now making our testing harness to make forall use random sampling
CODE6
The final step in this bug hunt is to write a bad implementation of sort.

8 Type Inference, Modules

We have a function and we want to identify it's type signature.

```
fun a -> (snd a, fst a);;
  snd: X * Y -> Y
  fst: X * Y -> X
```

First we must look at the top-level expression. We can express this as
$X? * Y? \rightarrow C? * D?$
Where $a : X? * Y?$
$snd(a) : Y?$
$fst(a) : X?$ We can substitute the functions in and fill in the polymorphic types
$f :' a *' b \rightarrow' b *' a$.
There is a website to practice polymorphic type inference but I didn't get the url.
We have the sample code

```
type 'a queue = 'a list
let size : 'a queue -> int = failwith "..."

type 'a q_ops = {
  push : 'a queue -> 'a -> 'a queue;
  peek : 'a queue -> 'a option;
}
```

This is a queue data structure which allows adding an element to the end and looking at the front. This is an example of a module?
In ocaml, files are modules. These are similar to a Java class with static methods.

```
let x : int Queue.t = create () ;;
Queue.add 1234 x;;
Queue.take_opt x;; (* int option = Some 1234 *)
```

Now to create a set, an unordered list of elements.

```
type 'a set = 'a list
let empty : 'a set = []

let rec mem (x : 'a) (s : 'a set) : bool
  = match s with
  | [] -> false
  | x' :: s' -> if x' = x then true else mem x s'

let rec remove (x : 'a) (s : 'a set) : 'a set
  = match s with
  | [] -> []
  | x' :: s' -> if x' = x the s' else x' :: remove x s'

let add (el : 'a) (s : 'a set) : 'a set = el :: s
```

We care about module and interface design in order to ensure usability. When storing a set as a list, we allow the user to call List. commands on it.
In order to prevent this we can define an interface file.

9 2/4

$f :' a.'a \to' a$ is a universal quantifier $\forall$.
A module is equivalent to the existensial quantifier $\exists$. We are stating that there exists an instance of a type that follows a proposition, in this case the proposition is our interface.

When creating a module interface, you can comment out a function to make a private method.

We can create module types in files:

```
module type MyIntSetSig = sig
  type t
  val empty : t
  val mem : int -> t -> bool
  val add : int -> t -> t
  val remove : int -> t -> t
```

We use t for our type in order to unify the output type for all of our methods.

```
module type MyIntSet : MyIntSetSig = struct
  type t = int list
  let empty : t = []
  let mem : int -> t -> bool = ...
  val add : int -> t -> t = ...
  val remove : int -> t -> t = ...
```

This struct signature defines a structed module.
We can also open up the scope of the module using a "scoped open". We can pass in a statement (such as a function into the instance of the module).

```
let ex1 = MyIntSet.(add 1 (add 2 ex0))
```

We can design a recursive random generator for random sets.

```
let rec build_set_acc (n : int) (acc : MyIntSet.t) : MyIntSet.t
  = if n <= 0
    then acc
    else (build_set_acc (n - 1) (MyIntSet.add (Random.int 10)
       acc))

let build_set (n : int) = build_set_acc n MyIntSet.empty
```

Even though this module holds an int list, we cannot call int functions on it because it is abstract.
Some say that interface design is the most important part of programming so we need to make meaningful choices in their creation.
If we supply a large collection of helper functions in an interface the type somewhat loses semantic meaning. Specifiying very few functions makes the interface simpler and easier to read, but inflexible.

```
module type StringSetSig = sig
  type t
  val empty : t
  val mem : string -> t -> bool
  val add : string -> t -> t
```

```
  val remove : string -> t -> t
end

module HashSet : StringSetSig = struct
  type t = string list
  let empty = []
  let add x xs = Digest.string x :: xs
  let mem x xs = List.mem (Digest.string x) xs
  let remove x xs = List.filter ((<>) (Digest.string x)) xs
end
```

We can use these abstract types to reason about invariance. Our interface can make specifications such as a bijection between two types.

```
type bo = unit option
```

This specifies an isomorphism between the booleans and unit option.

```
module type BSig = sig
  type t

ende
```

10 Invariants and Generalizing Modules

We can create the type Even with the emergent inavariant property $\forall(n : even), n \bmod 2 = 0$

We can equip our struct with a well-formedness property that ensures the invariance holds. This pushes some responsibility to the implementer and requires some hacking for testing.

Instead we can define a CounterSig module and corresponding Counter that can be a fresh variable and "counts" how many times it gets bumped.

This counter lacks the ability to actually count as we do not have mutability. We can instead define CounterWithId with t as a record type that uses unique ids.

We currently have our count˙evens function that can only work on Counter types but we might want to ma

For example:

```
let read_network_string () : string = ...
let read_local () : string = ...

let process () : int =
  let s = read_network () in
  if s = "hello" the 0 else 1
```

This is an example of an application where we might want to make our process either read network or local. The desired solution to this problem is to parameterize our read function i.e.

```
let process (read: unit -> string) : int =
  let s = read () in
  if s = "hello" the 0 else 1
```

To extend this to modules we need a wrapper module. These are called functors (which are different from all other things called functors)

We can now write properties such as $\forall$ *(c : CounterSig), get (bump c) = get c + 1*

There are some strange properties with polymorphism in this context. In the past we worked with MySetSig. We are reliant on the definition of equality where $\{(3,2)(2,3)\} = \{(2,3)\}$

Our solution is to design a new definition of equality, a functor called EqType that has the equals property. Now we can parameterize our set struct with EqType, using a second type elem = T.t

We can now create UnorderedPair with an equals function. Even though we cannot annotate the type, the compiler infers this based on structure.

11 Functors and Mutability

11.1 Functors, cont.

```
module type SetSig = SetSig
  type elem
  type t
  val empty : t
  val add : elem -> t -> t
  val mem : elem -> t -> bool
end
```

This is a functor that allows an abstract set type. We can implement it using this struct

```
module ListSet (* : SetSig *) = struct
  type elem = int
  type t = elem list
  let empty = []
  let add x xs = x :: xs
  let rec mem x xs =
    match xs with
    | [] -> false
    | y :: ys -> if x = y then true else mem x ys
end
```

The simple equality function is baked into OCaml, but equality is definitely not nontrivial especially from the perspective of type theory.

We can instead parameterize (as shown earlier) with an equality function.

```
module type EqType = sig
  type t
  val equals L t -> t -> bool
end

module ListSet (T : EqType) : (SetSig with type elem = T.t) =
   struct
  type elem = T.t
  type t = elem list
  let empty = []
  let add x xs = x :: xs
  let rec mem x xs =
    match xs with
    | [] -> false
    | y :: ys -> if T.equals x y then true else mem x ys
```

Now to create an implementation of EqType to determine string equality.

```
module StringEq : EqType = struct
  type t = string
  let equals s1 s2 = String.lowercase_ascii s1 = String.
     lowercase_ascii s2
end

module StringSet = ListSet(StringEq)
```

This is somewhat restrictive as we need to provide a new EqType every time we have an object. If we wanted to create a StringListEq, we can use functors in order to reuse our StringEq program.

```
module ListEq (T : EqType) (* : EqType *) = struct
  type t = T.t list
  let rec equals xs ys =
    match xs, ys with
    | [] , [] -> true
    | x :: xs' , y :: ys' -> T.equals x y && equals xs' ys'
    | _ , _ -> false
end

module StringListSet = ListSet(ListEq(StringEq))
```

In order to link the three types, we must add some extra type annotations.

```
  module ListSet (T : EqType) : (SetSig with type elem = T.t) =
     struct ...
  module StringEq : (EqType with type t = string) = struct ...
  module ListEq (T : EqType) : (EqType with type t = T.t list) =
     struct ...
```

11.2 Mutability

We have this code

```
let add_one x = x + 1

let f () =
  let x = 1 in
  let y = add_one x in
  let x = 7 in
  print_endline ("Y : " ^ string_of_int y)

let _ = f ()
```

This will output 2 as it ignores the reassignment x = 7. If we want to instead recompute x we can use reference types:

```
let add_one (x : int ref) = !x + 1
let f () =
  let x = ref 1 in
  let y = fun _ -> add_one x in
  x := 7;
  print_endline ("Y : " ^ string_of_int (y ()))

let _ = f ()
```

The new functions have signatures ref : $a' \to a'$ ref, ! $:' a\, ref \to' a$, and $(:=) :' a\, ref \to' a \to$ unit.
Included are a few excercises to learn how references work.

```
let test () =
  let f x =
```

```
    let y = ref 10 in
    y := !y + 1;
    x + !y
  in
  let y = ref 0 in
  f 2
```

This ignores the final let y and evaluates to 13.

```
  let f =
  let y = ref 0 in
    fun () ->
      y := !y + 1;
      !y
```

This counts up a value for y and increases each function call.

12 References and Aliasing

In OCaml, reference cells are type-invariant after they are created.
We can think as references as a pointer to a memory cell on the heap. Aliasing just closes the value with the same address.
When comparing two references with (=), the operator checks if their values are the same. We might want to treat them differently.

```
let f (x : int ref) (y : int ref) =
  y := 0;
  !x
```

This function is intended to:

- return contents of x
- modify y
- don't modify x

But when we call $let\ _ = f\ x\ x$ this will reset x which we don't want. This is a natural solution.

```
let f (x : int ref) (y : int ref) int option =
  if x = y
  then None
  else (y := 0; Some(!x))
```

This code is sound as it follows the requirements but will return None if we pass two different references with the same value.
Instead of structural equality, we need physical equality (==) to check if the addresses are the same.

```
let f (x : int ref) (y : int ref) int option =
  if x == y
  then None
  else (y := 0; Some(!x))
```

It becomes challenging to handle these things using logic.

- References may be mutated over test cases
- You must reason about the addresses and values
- Side effect that happens over time.

Monads??
In order to test the above function, we need to make a wrapper function with an inputed edge case.

```
let fwrapper (x : int) (y : int) (alias : bool)
: (int option * int * int) =
  let (xref, yref) =
    let xref = ref x in
    if alias
    then (xref, xref)
```

```
    else
      let yref = ref y in
      (xref, yref)
  in
  let o = (f xref yref) in
  (o, !xref, !yref)
```

The property now becomes

```
let prop_mut (ix, iy, alias) : bool =
  let (o, fx, fy) = fwrapper ix iy alias in
  if alias
  then o = None && ix = fx && iy = fy
  else
    match o with
    | None -> false
    | Some v -> v = fx && fy = 0 && fx = ix
```

We still have an issue with the side effect over time problem which we might want to test. It could be important to ensure that we are maintaining invariants dynamically.

13 2/25

Most applications are reactive and they can be described as **state machines**. A state machine is a set of states and transitions between them. A common example is a vending machine.
Some state machines have an unbounded amount of states, these can be represented by **process algebras**. These are important for reasoning about systems that change.

```
module type StateMachine = sig
  type state
  type input
  val init : state
  val step : state -> input -> state
end
(*
INIT := MACHINE(0.00, 10)
  MACHINE(money, items) :=
    insert25c.MACHINE(money + 0.25, items)
    + insert50c.MACHINE(money + 0.50, items)
    + vend.MACHINE(money - 1.00, items - 1) if money >= 1.00 and
       items >= 1
*)

module MoneyItemVendingMachine (* StateMachine *) = struct
  type state = { money: float; items: int }
  type input = Insert25c | Insert50c | Vend

  let init = { money= 0.0; items= 10}

  let step state input =
    match input with
    | Insert25c -> { state with money = state.money +. 0.25 }
    | Insert50c -> { state with money = state.money +. 0.50 }
    | Vend when state.money >= 1.00 && state.items >= 1 ->
      { money = state.money -. 1.00; items = state.items - 1}
    | Vend when state.money < 1.00 -> failwith "not enough money"
    | Vend when state.items < 1 -> failwith "out of stock"
    | Vend -> failwith "invalid state"
end
```

In order to test functions that change sequentially, we need a trace. Generate a sequence of inputs and consider properties about the state over the sequence.

We can write a functor that takes in a state machine and turns it into a machine trace. It takes in a list of inputs. The entire struct is an option as if an error is thrown we return **None**.

14 3/9

14.1 What is a program?

Definition 3. *A **Program** is a description of something computable.*

We have the question of _Syntax vs. _Semantics.
Often times we talk about programming with turing-complete systems but this is not always true. Take English for example.
English is comprised of sentences.

```
Sentence := (NounPhrase, VerbPhrase)
  NounPhrase := (determiner option, adjective option, noun)
  VerbPhrase := Simple of (verb, np option, adverb)
    | Rec (verb, VerbPhrase)
```

We have the sentence "Alice speaks" where Alice : noun and speaks : verb.
Another: "(a black cat)NP (can catch mice)VP"

This can be represented as a syntax tree `https://www.google.com/url?sa=t&source=web&rct=j&url=https%3A%2F%2Fwww.nltk.org%2Fbook%2Fch08.html&ved=0CBYQjRxqFwoTCIiemKmy`
`opi=89978449`
OCaml is also a language and also can be evaluated using a syntax tree.

14.2 Creating a Calculator Language

To create a language, you should start with examples of elements in the language.
We have Lang Calc. with "*", "+", and $n \in \mathbb{N}$.
This can be specified in an OCaml type.

```
type expr =
  | Nat of int
  | Add of expr * expr
  | Mul of expr * expr
```

We might want to make a serializer, a way to take this AST and turn it into a string.

```
let rec string_of_expr e =
  match e with
  | Nat i -> string_of_int i
  | Add (l, r) -> string_of_expr l ^ " + " ^ string_of_expr r
  | Mul (l, r) -> string_of_expr l ^ " * " ^ string_of_expr r
```

If we have the statement string˙of˙expr (Mul(Nat 3, Add(Nat 4, Nat 2))) we get "3 * 4 + 2", but this ignores the (4 + 2). We need a round-trip property:

```
let rec string_of_expr e =
  match e with
  | Nat i -> string_of_int i
  | Add (l,r) -> "(" ^ string_of_expr l ^ " + " ^ string_of_expr
     r ^ ")"
  | Mul (l,r) -> "(" ^ string_of_expr l ^ " * " ^ string_of_expr
     r ^ ")"
```

To parse a string in this language we create the token type

```
type token =
  | NAT of int
  | PLUS
  | TIMES
  | LPAREN
  | RPAREN
  | EOF
```

To specify our roundtrip property, $\forall e \in$ Calc, parse (string˙of˙expr e)

15 3/12 - Programming Languages, Cont.

We must make design choices when creating programming languages.
Currently, our language can do simple statements, but we now want to add binders:

```
let x = 2 in
let y = x * x in
y
```

There are a few ways to do this. Firstly, we must describe the kind of expressions we want. We also would like to be able to use free variables such as

```
let x = 5 in
y
```

This has three parts: the variable, body and the bind location. We have the functions Let("x", Nat Z, e1) and $x \to$ Var "x".

Note 1. *The next few parts of this lecture very heavily rely on the code from last class which I don't have :(*

16 3/16 - Intro to Type Checking

16.1 What is a Type

We might think of a type as a set, but this is not enough. A type represents a behavior predictibly.
Types came about because of Russel's Paradox, showing inconsistency of the set of sets.
We might define the structure $\{foo : int\}$ which nests the integers.
In our current interpreter, we have runtime errors when a function is given the wrong inputs.
We have the type $e : bool$ which is a collection of many things that finally end of as a boolean such as (if $1 = 2$ the false else false).
To add types we create:

```
type ty = TNat | TBool
```

In order to make this type-safe, we first need to make a typeof function. We don't want to throw a runtime error so instead we make our type checker an option that returns None if there is a type error.

17 3/23 - Compiling

Previously we looked at parse trees of OCaml files using ocamlc, but now we can do some operations on our tree.

1. -dlambda turns the tree into a racket-like lambda expression
2. -dinstr transforms the tree into low-level machine instructions
3. -S turns the tree into assembly

We have a pipeline from parsing (creating an AST) → typechecking → an intermediate representation with some optimizations → instructions that are runnable in some VM → bytecode or assembly which is runnable on your machine.
Today we are going to build a compiler, but first we must understand the instruction stage using a stack language.

Definition 4. *A **Stack Language***

We have our calc language and we want to convert elements to push instructions, i.e. (Calc: $2 \overset{compiling}{\rightarrow}$ [Push 2] $\mapsto 2 : []$)
Calc: 2 + 3 → [Push 2, Push 3, Apply Plus] $\mapsto 3 : 2 : [] \overset{"apply\ plus"}{\rightarrow} 5 : []$
$\langle$ "3 + 2", [], {}$\rangle$
$\mapsto \langle [Push3, Push2, App+], [], \{\}\rangle$
$\mapsto \langle [Push2, App+], 2 : [], \{\}\rangle$
$\mapsto \langle [App+], 3 : 2 : [], \{\}\rangle$
$\mapsto \langle [], 5 : [], \{\}\rangle$
Say we have $\langle$ "let x = 5 in x", [], {}$\rangle$
$\mapsto \langle [Push\ 5; Set\ "x"; Get\ "x"], [], \{\}\rangle$
$\mapsto \langle [Set\ "x"; Get\ "x"], 5 : [], \{\}\rangle$
$\mapsto \langle [Get\ "x"], 5 : [], \{x \mapsto 5\}\rangle$
$\mapsto \langle [], 5 : [], \{x \mapsto 5\}\rangle$ To create this we need to start with a semantic domain:

```
type value = int
```

As well as operations:

```
type op = | Add | Mul | Or | And | EQ
```

As well as some instructions:

```
type instr =
  | Push of value
  | PrimApp of op
  | Set of string
  | Get of string

  (* TODO: needed for ite *)
  | Jump of int
  | JumpIfNotZero of int
```

Note that we use strings for variable names which is not scalable.
To evalueate we first need some instructions:

```
let eval_op (op : op) (args : value list) : value =
  match (op, args) with
  | (Add, [n1; n2]) -> (n1 + n2)
  | (Mul, [n1; n2]) -> (n1 * n2)
  | (And, [b1; b2]) -> (if b1 = 1 && b2 = 1 then 1 else 0)
  | (Or, [b1; b2]) -> (if b1 = 1 || b2 = 1 then 1 else 0)
  | (LT, [n1; n2]) -> (if n1 < n2 then 1 else 0)
  | (EQ, [e1; e2]) -> (if e1 = e2 then 1 else 0)
  | _ -> failwith "runtime error: arity mismatch or unexpected
     argument"
```

18 Formalizing Compiler Correctness and Failiures of PBT

In the past, we have written a calculator language, a stack language, and a compiler to lower the first into the second.

Question 1. *How do we know a program in our calculator language is correct?*

We have a well-typed function that confirms the correctness of a program.

Question 2. *How do we ensure programs in the stack language are correct?*

Our compile function must be surjective ($f : A \to B, \forall b \in B, \exists a \in A, f(a) = b$). One way to define this within the context of our compiler is: $\forall(e : Calc)$, (eval-opt [] e : value option) = Some x $\implies$ Stack.run(compile e) = conv (eval [] e)

```
let eval_succeeds e : bool =
  try
    let v = eval Strmap.empty e in true
  with _ -> false

let conv v =
  match v with
  | Nat v -> v
  | Bool false -> 0
  | Bool true -> 1

let (=>) a b_thk = if a then b_thk () else true
let compiler_correct (e : CalcLang.expr) =
  (eval_succeeds e) =>
  (fun () -> execute (compile e) = conv (CalcLang.eval StrMap.
     empty e))
```

We have our statement of correctness, but we now need to do some extra steps for PBT:

1. A testing harness
2. Generators

```
let _ =
  let nsamples = 100 in
  match forall ~trials:nsamples gen_expr compiler_correct with
  | Some c -> print_endline "found counterexample"
  | None -> "Passes PBT"
```

This suggests that our compiler works but there are some issues with this assumption. There could be unfound counterexamples within the test cases. We also assume the correctness of the generators.
Because our data is tree-shaped and thus quite complex, it is quite difficult to generate relevant elements. With a nicer testing harness we would be able to see that the majority of our samples have depth ≤ 2.
One way to fix this is just by generating more samples but this does not scale. A better way to do this would be to inductively prove that it works.

19 4/1 - Compiler Optimization

In the previous lecture, we tested that the compiler was valid. Optimizing the compiler is similar.
A compiler has multiple phases:

1. parse
2. desugar
3. typechecking/inference
4. uniquify
5. lowering

But we could add in an optimization step that has a self loop

1. parse
2. desugar
3. typechecking/inference
4. uniquify
5. **codegen**
6. lowering

OCaml uses Superoptimization, a pass to turn a program into the smallest possible representation.
We plan to implement a compiler pass on the level of Calc in order to make our tree parsing more efficient. This code is built upon the languages from previous lectures.
In our CalcLang we have expressions that look like this:

```
let x = 1 + 2 in x
let ex0 = Let("x", App(Add, [Val (Nat 1); Val (Nat 2)]), Var "x")
```

One way we could optimize this is simply by performing the substitution and evaluation to get 3. We can start by reducing our statically identifiable operations.

```
let constfold_op (op, args) : CalcLang.expr =
  match op, args with
  | (Add, [Val (Nat n); Val (Nat m)]) -> Val (Nat (n + m))
  | (Add, [Val (Bool n); Val (Bool m)]) -> failwith "UNREACHABLE"
```

This works for this first simple case but it falls apart for the second one with a mistake. Though our typecheck should stop this but in the interest of simplicity we should learn how to deal with it seperately.

Instead of operating over expressions, we might want to pass in a function.

```
let constfold (e : expr) : expr =
  match e with
  | Val v ->
  | Var x ->
  | Let (x, e1, e2) -> Let (x, constfold e1, constfold e2)
  | App (op, args) ->
    (match op, args with
    | (Add, [Val (Nat n); Val (Nat m)]) -> Val (Nat (n + m))
    | (Mul, [Val (Nat n); Val (Nat m)]) -> Val (Nat (n * m))
    | (Or, [Val (Bool n); Val (Bool m)]) -> Val (Nat (n || m))
    (* simply follow the eval op for the rest *)
    | _ -> App (op, List.map constfold args))
  | ITE (g, t, f) -> ITE(constfold g, constfold t, constfold f)
```

This code only works in the case where there is only one operation. With compound operations (such as 1 + (2 * 3)) it will only apply once.
We want to continuously optimize our expression until we reach a fixed point.

```
let rec optimize (opt : expr -> expr) (e : expr) : expr =
  let e' = opt e in
  if e' = e
  then e
  else optimize opt e'
```

Once more we are faced with the problem of correctness. We could pass a trivial function (i.e. f(x)=0) but this doesn't preserve meaning.
We could define a property to state our pass is valid $\forall e : Calc, e = constfolde$.
We can also do some lazy boolean analysis:

```
  | ITE (Val (Bool true), t, f) -> constfold t
  | ITE (Val (Bool false), t, f) -> constfold f
```

Now to optimize substitution. In a let binder we are looking for a case that looks like $e'[x \mapsto e]$.

```
let rec substitute (x : string) (e : expr) (e' : expr) : expr =
  match e with
  | Val v -> Val v
  | Var x' -> if x' = x then e else Var x'
  | Let (x, e1, e2) ->
    if x = x' then Let (x, e1, e2)
    else Let (x', substitute x e1, substitute x e2)
  | App (op, args) -> App(op, List.map (fun arg -> substitute x e
      arg))
  | ITE (g, t, f) -> ITE(substitute x e g, substitute x e t,
     substitute x e f)
```

We can now change our constfold function to use this in the Let case:

```
  | Let (x, Val v, e2) -> substitute c (Val v) e2
  | Let (x, e1, e2) -> Let (x, constfold e1, constfold e2)
```